UNIVERSIDADE FEDERAL DE SERGIPE
DEPARTAMENTO DE COMPUTAÇÃO
DOCENTE: GLAUCO DE FIGUEIREDO CARNEIRO
SISTEMAS OPERACIONAIS
TURMA 02

VIRTUALIZAÇÃO, CONTÊINERES, NAMESPACES E CGROUPS

DISCENTES:

ANDERSON SOARES DE SANTANA JÚNIOR
ANTÔNIO CARLOS BISPO CUNHA
ANTÔNIO JOSÉ MARTINS MASCARENHAS
DAVI OLIVEIRA MACHADO
JOÃO FELIPE TUNES OLIVEIRA
JOSÉ GUSTAVO ABREU ALVES
KAIQUE TEIXEIRA RODRIGUES

URL DO VÍDEO: https://youtu.be/LrUAWQ2_1Oc?si=EF5r7Bb7mJylslcx

URL DO REPOSITÓRIO:
https://github.com/antoniomartins13/SO_UFS_2026_2_Mascarenhas_Antonio_Alves_Jose_AV3

SÃO CRISTÓVÃO-SE
10/11/2026

1. IDENTIFICAÇÃO E CONTRIBUIÇÕES

Tabela 1 – Contribuição individual

Nº	Integrante	Contribuição
1	Antônio José Martins Mascarenhas	Ambiente Docker, diagrama de infraestrutura, coleta dos namespaces (Partes A e B) e criação do repositório
2	Anderson Soares	Testes de cgroups com carga de CPU, montagem read-only e extração dos logs brutos (Partes C e D)
3	Antônio Carlos	Tabela comparativa de handles e explicação do PID 1
4	Kaíque Teixeira	Processamento dos logs de cgroups, trabalho por segundo e gráfico comparativo
5	José Gustavo Abreu Alves	Análise do teste de montagem; fundamentação sobre kernel compartilhado, throttling e segurança em inferência
6	João Felipe	Relatório PDF, repositório/README e declaração de uso de IA
7	Davi Machado	Tabela de contribuições, vídeo da equipe e slides

Fonte: Próprio Autor

2. OBJETIVO E PERGUNTA NORTEADORA

Este trabalho executou um laboratório pequeno e reprodutível para observar isolamento de processos e limites de recursos em contêineres Linux. O foco é demonstrar mecanismos do sistema operacional com comandos, dados e interpretação, e não reconstruir uma aplicação completa. Não foram usados modelo de linguagem, GPU nem RAG.

A pergunta norteadora é: como namespaces alteram a visão de processos, mounts e rede, e como cgroups limitam recursos sem transformar um contêiner em uma máquina virtual independente? Dela decorrem quatro objetivos específicos: (i) distinguir máquina virtual, hypervisor e contêiner a partir do ambiente efetivamente utilizado; (ii) identificar evidências de namespaces PID, mount e network em dois contêineres; (iii) configurar e confirmar quotas de CPU, teto de memória e limite de tarefas, comparando trabalho fixo sob duas quotas; e (iv) demonstrar a negação de escrita em uma montagem somente leitura, sem atacar recursos do host.

3. AMBIENTE E DIAGRAMA (PARTE A)

A coleta usou dois ambientes Linux, ambos com Docker Engine rootful, cgroup v2 e a mesma imagem. As Partes A e B foram executadas em Linux nativo; as Partes C e D, em uma VM WSL2. A Tabela 2 registra os dois, e a Figura 1 mostra as camadas de cada um.

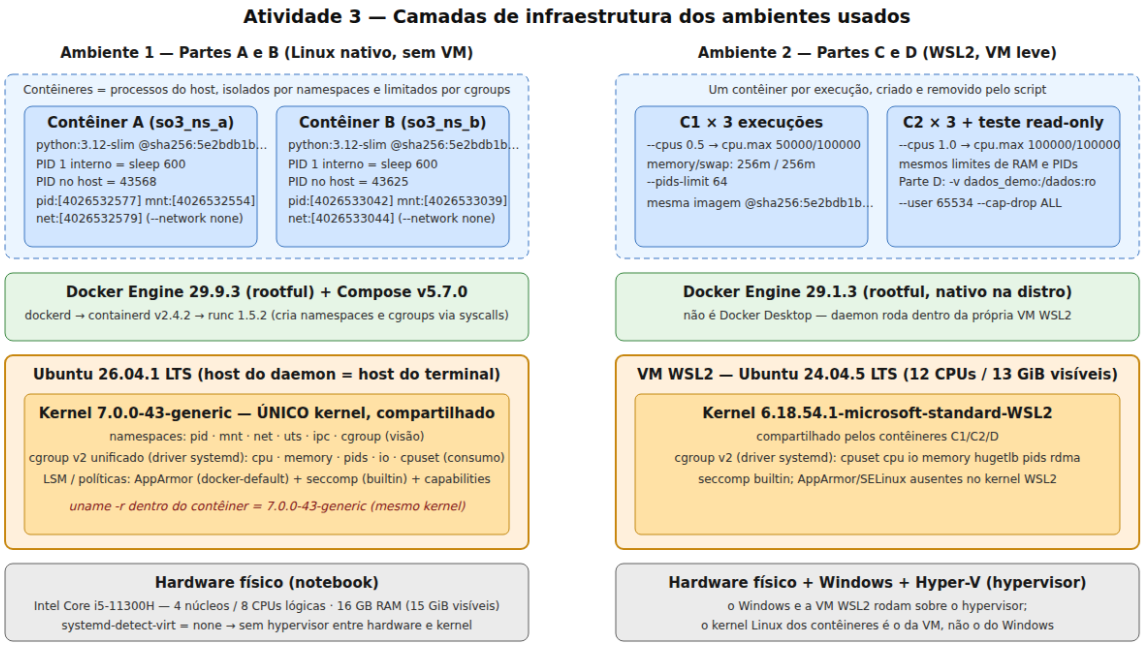
Tabela 2 – Ambientes de execução

Item	Partes A e B (namespaces)	Partes C e D (cgroups e read-only)
Host do daemon	Linux nativo, sem VM (systemd-detect-virt = none)	Ubuntu 24.04.5 LTS dentro do WSL2 (VM leve sobre Hyper-V)
Distribuição / kernel	Ubuntu 26.04.1 LTS; 7.0.0-43-generic	Ubuntu 24.04.5 LTS; 6.18.54.1-microsoft-standard-WSL2
CPUs lógicas / RAM	8 / 15 GiB (Intel Core i5-11300H)	12 / 13 GiB
Docker	Engine 29.9.3 (rootful); Compose v5.7.0; containerd v2.4.2; runc 1.5.2	Engine 29.1.3 (rootful, nativo na distro; não é Docker Desktop)
cgroup	v2, driver systemd	v2, driver systemd
Controladores na raiz	cpuset cpu io memory hugetlb pids rdma misc dmem	cpuset cpu io memory hugetlb pids rdma
LSM	AppArmor (default) e seccomp (builtin)	seccomp (builtin); AppArmor e SELinux ausentes

Fonte: Próprio autor

A imagem é a mesma nos dois ambientes, fixada por digest: python@sha256:5e2bdb1b0d542d5039cffb8181c65930093949194a65c365dfb1b0fe13a4fa5b (python:3.12-slim). Executado com as flags da condição C1, o próprio contêiner leu em /sys/fs/cgroup os valores cpu.max = 50000 100000, memory.max = 268435456, memory.swap.max = 0 e pids.max = 64, com cgroup namespace privado (raiz vista: 0::/). Dentro do contêiner, o comando uname -r retornou 7.0.0-43-generic, o mesmo kernel do host.

Figura 1 – Camadas de infraestrutura dos dois ambientes



Fonte: Próprio Autor

Onde está o kernel compartilhado? Há VM intermediária? Contêineres não têm kernel próprio: fazem syscalls ao kernel Linux do host do daemon. No ambiente nativo, não há VM entre o hardware e o kernel. No ambiente WSL2, há uma VM: o kernel compartilhado pelos contêineres é o da VM, não o do Windows, que fica abaixo, junto ao hypervisor, e influencia o escalonamento e a frequência da CPU.

4. EVIDÊNCIAS DE NAMESPACES (PARTE B)

Foram executados dois contêineres independentes da mesma imagem (so3_ns_a e so3_ns_b), com --network none, --cap-drop ALL, --security-opt no-new-privileges e o comando sleep 600. A coleta oficial é de 10/11/2026, com as leituras do host feitas como root. Uma coleta anterior (09/11/2026), feita sem root, foi preservada em evidencias/namespaces_coleta1.txt: nela não foi possível ler /proc/<PID>/ns/* de fora, e os resultados qualitativos eram os mesmos. Os handles foram lidos com docker exec NOME readlink /proc/1/ns/{pid,mnt,net}, e o PID do host, com docker inspect. A Tabela 3 compara os resultados.

Tabela 3 – Handles de namespaces dos contêineres A e B

Recurso	Handle em A	Handle em B	Host	A × B	Interpretação
PID	pid:[4026532577]	pid:[4026533042]	pid:[4026531836]	Diferentes	Árvores de PID distintas
mount	mnt:[4026532554]	mnt:[4026533039]	mnt:[4026531832]	Diferentes	Tabelas de montagem próprias
network	net:[4026532579]	net:[4026533044]	net:[4026531833]	Diferentes	Pilhas de rede separadas

Fonte: Próprio Autor

Pela namespaces(7), dois processos estão no mesmo namespace se, e somente se, o handle (número de inode no nsfs) é idêntico. Aqui todos diferem entre A, B e o host, de modo que cada um tem visão distinta de processos, montagens e rede. A coluna do host vem do PID 1 do host (systemd), lido como root; o shell que executou o script tem exatamente os mesmos handles, o que confirma que ambos estão no namespace inicial. Com --network none, cada contêiner enxerga apenas a interface lo, com contadores zerados.

A ligação entre dentro e fora foi verificada: para cada contêiner e cada tipo, readlink /proc/<PID do host>/ns/<tipo>, lido no host como root, deu o mesmo handle que readlink /proc/1/ns/<tipo> dentro do contêiner (6 de 6 marcados como IGUAL no log), e o lsns do host lista cada um desses namespaces com um processo, o respectivo sleep 600. O kernel reutiliza o número do inode depois que um namespace é destruído: por isso o handle mnt:[4026532554] também aparece na coleta anterior, cujos contêineres já tinham sido removidos.

4.1 Por que ambos têm PID 1?

Dentro de A e de B, /proc/1/comm retorna sleep: os dois contêineres têm um “PID 1”, mas não são o mesmo processo. No host, o de A é o PID 43568 e o de B é o PID 43625, ambos sleep 600 com usuário root, e cada um é filho do seu próprio containerd-shim-runc-v2 (PIDs 43543 e 43601), o processo que o runtime deixa como pai do contêiner. A linha NSpid: 43568 1 em /proc/43568/status mostra que o mesmo processo tem dois números: 43568 no namespace PID do host e 1 no do contêiner (e, analogamente, 43625 e 1 para B). Um namespace PID novo reinicia a numeração, e o primeiro processo criado nele é o init daquele namespace, embora continue existindo no namespace pai com um PID próprio. Os handles pid diferentes, confirmados de dentro e de fora, explicam por que o número 1 se repete

sem conflito: a unicidade do PID vale dentro de cada namespace. O sh de PID 39 é o processo criado pelo docker exec, que entra (setns) no namespace já existente.

Como o daemon roda no próprio Linux nativo, o PID informado pelo docker inspect pôde ser conferido no /proc do terminal. Em Docker Desktop ou em WSL2 com o daemon em outra VM, essa verificação não seria válida.

4.2 O que o mountinfo mostra

O /proc/1/mountinfo de cada contêiner mostra que a raiz é um sistema overlay (mount ID 1650 em A e 1818 em B). As camadas inferiores dos snapshots 56 a 59, que correspondem à imagem, são as mesmas nos dois e são somente leitura; cada contêiner tem a própria camada gravável (upperdir 73 em A e 75 em B). A primeira camada inferior também difere (72 em A, 74 em B), provavelmente a camada de init criada pelo Docker, o que o log não confirma. Em consequência, a imagem é compartilhada em disco, mas o que um contêiner escreve não vai para o outro. Além disso, /sys e /sys/fs/cgroup aparecem montados como ro nos dois.

Não foi feita uma montagem nova dentro de um contêiner para verificar se ela apareceria no outro: com --cap-drop ALL não há CAP_SYS_ADMIN, e isso não seria possível sem enfraquecer o teste. A separação de montagens entre namespaces fica, portanto, como mecanismo descrito em namespaces(7) e mount_namespaces(7), e não como observação direta.

4.3 O que os handles não demonstram

Handles diferentes provam que os namespaces são distintos, isto é, que a visão de PIDs, mounts e rede é separada. Eles não provam isolamento de segurança absoluto, porque o kernel é compartilhado e uma falha nele ou no runtime pode ultrapassar os namespaces; nem limite de consumo, que é função dos cgroups; nem restrição de privilégios, que depende de identidade, capabilities e políticas como seccomp e AppArmor. Em uma frase: namespaces mudam o que o processo vê, e cgroups, identidade e políticas de acesso mudam o que ele pode consumir e fazer. Esta evidência vale para o ambiente Linux nativo das Partes A e B; os experimentos de cgroups (Partes C e D) foram feitos em WSL2 e não devem ser tratados como observação do mesmo ambiente.

5. METODOLOGIA (PARTES C E D)

A carga é determinística e de trabalho fixo: a soma de i^2 módulo $(2^{61}-1)$ para $i < N$, em uma única thread, com checksum conferido pela forma fechada $N(N-1)(2N-1)/6$. O script abandona a carga sozinho após 60 s, e o comando é limitado a 90 s no host. Um script executado dentro do contêiner (medir.py) lê /sys/fs/cgroup antes e depois da carga e amostra memory.current a cada 0,2 s. Os scripts foram implementados pela equipe, equivalentes ao kit de apoio, que não estava disponível localmente.

Flags comuns às duas condições: --memory 256m, --memory-swap 256m, --pids-limit 64, --cgroupns private, --network none, --user 65534:65534, --cap-drop ALL, --security-opt no-new-privileges e --read-only. A condição C1 usou --cpus 0.5 e a C2, --cpus 1.0. Um piloto com N = 2.000.000 durou cerca de 0,13 s de CPU, curto demais; com N = 100.000.000, as execuções ficaram em cerca de 16 s (C1) e 8,6 s (C2), abaixo do limite. Esse valor foi congelado para as seis execuções principais, realizadas em ordem alternada: C1 r1, C2 r1, C2 r2, C1 r2, C1 r3, C2 r3. Nenhuma execução falhou nem foi descartada.

O limite efetivo foi confirmado por leitura de cpu.max, memory.max, memory.swap.max e pids.max dentro do cgroup, e não pela flag solicitada; docker inspect registra apenas o que foi pedido. Por execução, foram coletados trabalho concluído por segundo (que não é tokens por segundo), wall time, tempo de CPU do processo, deltas de cpu.stat (usage_usec, nr_periods, nr_throttled e throttled_usec) e três medidas distintas de memória: o RSS de pico do processo (ru_maxrss), o memory.current amostrado do grupo e o memory.peak desde a criação do cgroup.

Na Parte D, o diretório sintético dados_demo, com o arquivo texto.txt (conteúdo: Documento sintetico), foi montado em /dados. Nada além dele foi montado (nem a raiz, o home ou o socket do Docker); não se usou --privileged nem se desativou qualquer LSM. Os casos foram: D1, usuário nobody (UID 65534) com mount ro; D2, root (UID 0) sem capabilities com mount ro; e D3, nobody com mount rw sobre uma cópia sintética, como controle. Todos usaram --network none, --cap-drop ALL, --security-opt no-new-privileges, --read-only, --cpus 0.5, --memory 64m e --pids-limit 16. O hash SHA-256 e a listagem do diretório no host foram registrados antes e depois.

6. RESULTADOS

6.1 Parte C: C1 × C2

A Tabela 4 traz as seis execuções válidas. Em todas, o checksum coincidiu com o esperado (6629214017524433440) e a carga concluiu as 100.000.000 iterações.

Tabela 4 – Execuções principais (N = 100.000.000 iterações)

Exec.	Wall (s)	CPU proc. (s)	Mi iter/s	Δnr_periods	Δnr_throttled	Δthrottled (s)	peak (MiB)
C1 r1	15,79	7,87	6,33	159	159	7,96	17,9
C1 r2	16,07	8,01	6,22	162	162	6,86	18,3
C1 r3	16,09	8,02	6,22	162	162	6,89	18,2
C2 r1	8,39	7,74	11,91	84	0	0,00	18,2
C2 r2	8,43	7,78	11,86	85	0	0,00	18,5
C2 r3	10,16	9,37	9,85	103	0	0,00	17,9

Fonte: Próprio autor.

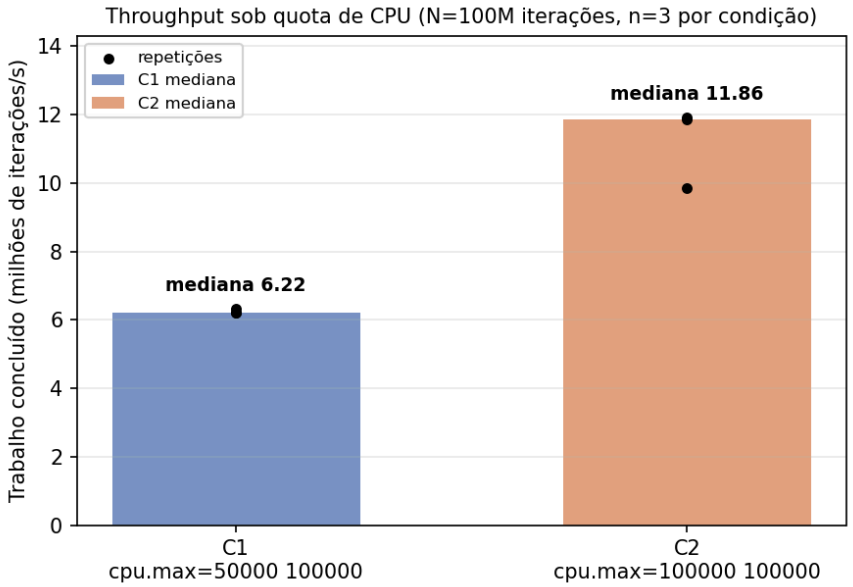
Tabela 5 – Consolidado por condição (n = 3)

Indicador	C1 (0,5 CPU)	C2 (1,0 CPU)
cpu.max confirmado	50000 100000	100000 100000
Mediana de trabalho (Mi iter/s)	6,22	11,86
Mín–Máx (Mi iter/s)	6,22–6,33	9,85–11,91
Coeficiente de variação	1,05%	10,52%
Wall mediano (s)	16,07	8,43
CPU do processo mediano (s)	8,01	7,78
nr_throttled (mediana)	162	0
RSS de pico do processo (MiB)	15,2	15,1
memory.peak do cgroup (MiB)	18,2	18,2

Fonte: Próprio autor.

Em todas as execuções, pids.peak foi 6 (limite 64, sem eventos pids.events max) e não houve oom_kill. A Figura 2 mostra o gráfico comparativo.

Figura 2 – Trabalho concluído por segundo sob quota de CPU (N = 100M iterações, n = 3 por condição)



Fonte: Próprio autor.

6.2 Parte D: montagem somente leitura

Tabela 6 – Leitura e tentativa de criação de arquivo em /dados

Caso	Identidade	Mount	Leitura	Criar arquivo	Mecanismo
D1	nobody (65534)	ro	OK	Negada: EROFS (errno 30)	flag ro
D2	root (0), sem capabilities	ro	OK	Negada: EROFS (errno 30)	flag ro
D3	nobody (65534)	rw	OK	Negada: EACCES (errno 13)	permissão DAC

Fonte: Os autores (2026), a partir de resultados/parte_d/D1–D3.

Os três logs registram CapEff = 0, CapBnd = 0, NoNewPrivs = 1 e Seccomp = 2. O mountinfo mostra /dados com ro,relatime em D1 e D2, enquanto o superbloco ext4 do host continua rw. O hash SHA-256 do arquivo (e02438…d5d7) e a listagem do diretório no host foram idênticos antes e depois: nada foi criado nem alterado.

7. ANÁLISE

7.1 Como a quota foi confirmada? Houve throttling?

A quota foi confirmada pela leitura de cpu.max dentro do cgroup. Em C1, houve throttling em 100% dos períodos: Δnr_throttled foi igual a Δnr_periods (159, 162 e 162) nas três repetições. Os contadores fecham com o tempo e o trabalho: cerca de 8 s de CPU em cerca de 16 s de parede dão razão de aproximadamente 0,50, isto é, 50.000 µs a cada 100.000 µs. O processo não roda mais devagar:

executa em velocidade normal por cerca de 50 ms e é suspenso até o próximo período. Em C2, Δnr_throttled foi zero, porque a carga tem uma única thread e não consegue consumir mais de uma CPU por período. A quota é tempo agregado por período, e não afinidade nem reserva de núcleo: o processo pode migrar entre as CPUs lógicas, e o cpuset não foi alterado.

7.2 Mediana e variação

A mediana de trabalho por segundo foi de 6,22 milhões de iterações por segundo em C1 e 11,86 em C2, razão de 1,905: próxima de 2, mas não idêntica. O tempo de CPU necessário para o trabalho fixo foi praticamente o mesmo nas duas condições (cerca de 7,8 a 8,0 s), o que mostra que a quota altera quando o processo pode executar, e não quanto trabalho custa. C1 foi muito estável (CV de 1,05%) porque o gargalo é determinístico. C2 teve uma repetição lenta (r3, 9,85 Mi iter/s) sem nenhum throttling, e o tempo de CPU do próprio processo também subiu (9,37 s contra cerca de 7,8 s). Isso aponta para fatores externos ao cgroup, como frequência e turbo da CPU, vizinho SMT e atividade do Windows sobre a VM WSL2, mas essas hipóteses não foram isoladas, e uma única amostra não permite afirmar a causa.

7.3 Principal ameaça à validade

A principal ameaça é o host compartilhado e não controlado: o WSL2 é uma VM cujo escalonamento e frequência de CPU dependem do Windows, e não se fixou frequência nem se isolaram núcleos. Somam-se a isso apenas três repetições por condição, uma única carga sintética (CPU-bound, single-thread, em Python) e o fato de que o tempo de partida do interpretador entra no Δusage do cgroup, mas não no wall da carga. O resultado não deve ser extrapolado para geração de tokens nem para inferência com GPU.

7.4 Três medidas de memória

O RSS de pico do processo (cerca de 15,2 MiB) conta apenas as páginas residentes do processo da carga. O memory.current amostrado (máximo de cerca de 18 MiB) é o uso instantâneo de todo o cgroup, incluindo page cache e memória de kernel cobrada. O memory.peak (cerca de 18 a 18,5 MiB) é o máximo de

memory.current desde a criação do cgroup. Nenhuma delas é “RAM média”, e todas ficaram muito abaixo do teto de 256 MiB.

7.5 Namespace, cgroup, identidade e política de acesso

O namespace de mount dá ao contêiner sua própria árvore de montagens e define o que é visível, não o que é gravável. A montagem ro é uma flag por ponto de montagem: o VFS recusa a escrita com EROFS antes de checar permissões, independentemente do usuário, e por isso até o root de D2 foi bloqueado. A identidade e as permissões (DAC) usam UID, GID e modo: em D3 o mount permitia escrita, mas o usuário nobody não era dono do diretório, e a negação veio com EACCES. Segundo o README das Partes C e D, o diretório pertencia ao UID 1000 com modo 755; o log de D3 lista apenas o arquivo, de modo que essa informação vem do README. Capabilities, no-new-privileges e seccomp reduzem operações privilegiadas: sem CAP_DAC_OVERRIDE e CAP_SYS_ADMIN, o root do contêiner não ignora permissões nem remonta /dados como rw. O cgroup é outro eixo: limita quanto se consome e não controla acesso a arquivos.

A negação de escrita não prova isolamento absoluto. O kernel é compartilhado, o daemon é rootful e o conteúdo permanece legível. No ambiente da Parte D, o perfil LSM lido em /proc/self/attr/current foi “kernel”, ou seja, AppArmor e SELinux não estavam ativos, e a contribuição dessa camada não pôde ser observada.

8. SEGURANÇA EM INFRAESTRUTURA COMPARTILHADA DE INFERÊNCIA

Em um servidor de inferência compartilhado, a quota de CPU evita que um job, como o pré ou pós-processamento, monopolize o processador e faça os vizinhos esperarem. O memory.max contém o consumo excessivo de RAM, por exemplo por um contexto muito grande ou um vazamento: o OOM fica restrito ao cgroup em vez de afetar o host. O pids.max limita a proliferação de processos e threads. Nos testes, os limites foram memory.max de 256 MiB, memory.swap.max igual a 0 e pids.max igual a 64, com pico de cerca de 18 MiB e 6 tarefas, sem eventos de OOM.

Montar o corpus, o diretório de modelos ou os pesos como somente leitura impede que um processo comprometido, ou um erro, altere esses artefatos por aquele ponto de montagem, protegendo a integridade contra, por exemplo, o

envenenamento de dados. Com identidade não root, capabilities removidas e no-new-privileges, aplica-se o princípio do menor privilégio: o serviço recebe apenas os arquivos e as operações de que precisa.

Alguns riscos permanecem. A montagem ro não protege a confidencialidade: os dados continuam legíveis e podem ser copiados, o que justifica também o uso de --network none no teste e o controle de saída de rede em produção. O kernel e o runtime continuam compartilhados, e uma falha neles contorna namespaces, cgroups e ro; por isso é preciso manter ambos atualizados. Os cgroups não isolam cache L3, banda de memória nem frequência compartilhada, o chamado vizinho barulhento, e a variação observada em C2 é consistente com isso, sem ser prova. A memória RAM não é VRAM: o memory.max não limita memória de GPU. Por fim, o ro vale para aquele ponto de montagem, e outro caminho com acesso de escrita aos mesmos arquivos o contorna.

9. LIMITAÇÕES

Os experimentos usaram dois ambientes diferentes (a coleta de namespaces, feita primeiro sem root e refeita com root, está preservada nos dois arquivos), e os números de cgroups e a Parte D referem-se somente ao WSL2, que introduz uma VM cujo desempenho depende do Windows; frequência e núcleos não foram fixados. A amostra tem três repetições por condição e uma única carga sintética, de uma thread, e não houve testes de significância. O Docker era rootful, e AppArmor e SELinux estavam ausentes no ambiente da Parte D. Os logs da Parte D contêm o caminho local do host do daemon, que deve ser revisado antes da publicação do repositório, assim como os arquivos de inspect, conforme orienta o enunciado.

10. CONCLUSÃO

Os dados sustentam que a quota de 0,5 CPU causou throttling em todos os períodos e reduziu a taxa de trabalho, enquanto a quota de 1,0 CPU, sem throttling, entregou mediana 1,91 vez maior, e não exatamente 2, por ser limitada pela velocidade de um núcleo e pelo ruído do host. As evidências de namespaces mostraram handles distintos de PID, mount e network entre dois contêineres, e a Parte D confirmou leitura permitida e escrita negada no bind mount ro, inclusive para root sem capabilities (EROFS), distinguindo esse mecanismo da negação por

permissões (EACCES). Namespaces, cgroups, identidade e políticas de acesso cumprem funções complementares: reduzem riscos, mas não transformam o contêiner em uma máquina virtual independente nem eliminam os riscos de um kernel e de um hardware compartilhados.

11. USO CRÍTICO DE IA GENERATIVA

A Tabela 7 declara as ferramentas, a finalidade, o destino da sugestão e a verificação. Nenhuma saída de IA foi apresentada como medição: todos os números deste relatório vêm dos registros brutos do repositório.

Tabela 7 – Declaração de uso de IA

Ferramenta	Finalidade	Sugestão	Verificação
Claude Code (Anthropic)	Scripts das Partes C e D (carga_cpu.py, medir.py, run_experimentos.sh, resumo.py, parte_d_readonly.sh), equivalentes ao kit de apoio	Adotada, após execução real dos scripts	Execução real; checksum pela forma fechada; cpu.max lido no cgroup
Claude (Anthropic)	Verificação de consistência entre os materiais da equipe e redação deste relatório	Adotada, com revisão pela equipe	Valores recalculados e conferidos com execucoes.csv, resumo.md, logs e evidências de A e B

Fonte: Os autores (2026).

REFERÊNCIAS

DOCKER. **Bind mounts**. Docker Docs. Disponível em: https://docs.docker.com/engine/storage/bind-mounts/. Acesso em: 9 nov. 2026.

DOCKER. **Resource constraints**. Docker Docs. Disponível em: https://docs.docker.com/engine/containers/resource_constraints/. Acesso em: 9 nov. 2026.

LINUX KERNEL DOCUMENTATION. **Control Group v2**. Disponível em: https://docs.kernel.org/admin-guide/cgroup-v2.html. Acesso em: 9 nov. 2026.

LINUX MAN-PAGES PROJECT. **namespaces(7)**. Disponível em: https://man7.org/linux/man-pages/man7/namespaces.7.html. Acesso em: 9 nov. 2026.